Session 9

1. Create a SQL script to insert a new order into an Orders table, then use COMMIT to save the transaction and verify that the new order persists after reconnecting to the database.

➔

```
USE db_restaurant;

CREATE TABLE orders (
    order_id INT(11) AUTO_INCREMENT PRIMARY KEY,
    user_id INT(11),
    item VARCHAR(255),
    amount DECIMAL(8,2)
);

START TRANSACTION;

INSERT INTO orders(user_id,item,amount)
VALUES
(101,'Veg Thali',130.00),
(102,'Punjabi Thali',150.00),
(103,'Rajasthani Thali',180.00);

COMMIT;

SELECT * FROM orders;
```

Output | Result #6

order_id	user_id	item	amount
1	101	Veg Thali	130.00
2	102	Punjabi Thali	150.00
3	103	Rajasthani Thali	180.00

2. Simulate a Zomato-style food order process: insert two new items into an Orderitems table, then use ROLLBACK to undo the changes before committing. Check that no new items remain in the table after rollback.

➔

```
USE db_restaurant;

CREATE TABLE orderitems(
   item_id INT(11) PRIMARY KEY AUTO_INCREMENT,
   item_name VARCHAR(255),
   price DECIMAL(8,2)
);

START TRANSACTION;

INSERT INTO orderitems(item_name,price)
VALUES
('Panner Butter Masala',280.00),
('Garlic naan',70.00);

SELECT * FROM orderitems;

ROLLBACK;

SELECT * FROM orderitems;
```

➔ Before Rollback

Output | Result #5 | Result #7

item_id	item_name	price
1	Panner Butter Masala	280.00
2	Garlic naan	70.00

➔ After Rollback

Output | Result #5 | Result #7

item_id	item_name	price

3. In a Flipkart-like shopping cart scenario, use SAVEPOINT to mark a point after adding a product to the Cart table, add another product, then use ROLLBACK TO SAVEPOINT to undo only the last addition. Show the final contents of the Cart table.

➔

```
USE db_restaurant;

CREATE TABLE cart(
    cart_id INT(11) PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(255),
    price DECIMAL(8,2)
);

START TRANSACTION;

INSERT INTO cart(product_name,price)
VALUES('Smart Phone',22000.00);

SAVEPOINT after_first_product;

INSERT INTO cart(product_name,price)
VALUES('Laptop',72000.00);

ROLLBACK TO SAVEPOINT after_first_product;

COMMIT;

SELECT * FROM cart;
```

Output | Result #9

cart_id	product_name	price
1	Smart Phone	22000.00

4. Write a trigger on a Wallet table that automatically deducts the purchase amount from the user's balance whenever a new transaction is inserted, similar to how Paytm updates wallet balance after a payment.

➔

```
USE pl_sql;

CREATE TABLE Wallet (
    user_id INT PRIMARY KEY,
    balance DECIMAL(10,2)
);

CREATE TABLE Transactions (
    txn_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    amount DECIMAL(10,2),
    FOREIGN KEY (user_id) REFERENCES Wallet(user_id)
);

INSERT INTO Wallet (user_id, balance)
VALUES (1, 5000.00), (2, 3000.00);

DELIMITER &&

CREATE TRIGGER deduct_balance
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    UPDATE Wallet
    SET balance = balance - NEW.amount
    WHERE user_id = NEW.user_id;
END

DELIMITER ;


INSERT INTO Transactions (user_id, amount)
VALUES (1, 800.00);

SELECT * FROM Wallet;
```

Output | Result #9

user_id	balance
1	4200.00
2	3000.00

5. Use ChatGPT to generate a SQL test scenario where a trigger incorrectly updates a user's balance after an expense entry. Paste the scenario and your corrected trigger code, explaining how you fixed the error. Hint: Ask ChatGPT for a buggy trigger example and a test case that exposes the bug.

➔

SQL Trigger Bug – Expense Balance Update

1. Scenario

When a user adds an expense, a SQL trigger automatically deducts the expense amount from their balance.

But due to a mistake in the trigger, the amount gets deducted twice and affects all users.

2. Buggy Trigger Code

```
None
DELIMITER //

CREATE TRIGGER expense_update
AFTER INSERT ON Expenses
FOR EACH ROW
BEGIN
    UPDATE Users
    SET balance = balance - NEW.amount;

    UPDATE Users
    SET balance = balance - NEW.amount
    WHERE user_id = NEW.user_id;
END //

DELIMITER ;
```

Error: The first UPDATE has no WHERE condition, so it deducts money from every user.

3. Test Scenario

Suppose we have these users:

User	Initial Balance
Jay	₹10,000
Rahul	₹5,000

Add an expense of ₹500 for Jay.

```
None
INSERT INTO Expenses (user_id, amount)
VALUES (1, 500);
```

Expected Result:

User	Balance
Jay	₹9,500
Rahul	₹5,000

Buggy Result:

User	Balance
Jay	₹9,000
Rahul	₹4,500

4. Corrected Trigger Code

```
None
DELIMITER //

CREATE TRIGGER expense_update
AFTER INSERT ON Expenses
FOR EACH ROW
BEGIN
    UPDATE Users
    SET balance = balance - NEW.amount
    WHERE user_id = NEW.user_id;
END //

DELIMITER ;
```

5. How I Fixed the Error

1. Removed the extra UPDATE statement.
2. Added the WHERE condition to update only the correct user.
3. Used `NEW.amount` to get the newly added expense amount.

6. Conclusion

The trigger was deducting the expense twice and affecting all users. I fixed it by using only one UPDATE statement with a proper WHERE condition.

Now, only the user's balance who added the expense gets updated correctly.